PROCESAMIENTO FUNCIONAL DE DATOS PARA UN SISTEMA DE GESTIÓN DE TRANSPORTE URBANO

Juan Miguel Guzmán

Introducción

El caso TecnoMovil Data plantea modernizar el procesamiento de información del transporte urbano. La plataforma recibe registros de validaciones, estaciones y rutas, por lo que se requiere procesar colecciones de datos de forma clara y eficiente. En este trabajo se construye una solución académica en Java que usa programación funcional para analizar datos simulados de pasajeros.

Objetivos

Objetivo general

Desarrollar un módulo Java que procese registros simulados de transporte urbano mediante Streams, lambdas, funciones puras y colecciones tratadas de forma inmutable.

Objetivos específicos

• Representar las validaciones de pasajeros con una clase de datos inmutable.

• Aplicar filtros, agrupaciones, ordenamientos y reducciones para obtener indicadores.

• Explicar el uso de funciones puras, inmutabilidad, funciones de orden superior, lambdas y Streams.

• Comprobar la ejecución y organizar el proyecto para publicarlo en GitHub.

Desarrollo

Descripción de la solución

El proyecto contiene cuatro clases: Registro.java representa cada validación; GeneradorDatos.java crea 12.000 registros simulados; ProcesadorTransporte.java contiene las operaciones funcionales; y Main.java ejecuta los análisis y muestra los resultados.

Estructura de los datos

Cada registro contiene idUsuario, ruta, estación, acción (entrada o salida) y timestamp. La fecha se representa con LocalDateTime. Los datos son simulados porque el caso de estudio no proporciona una base de datos real.

Operaciones implementadas

a) Afluencia por estación

Filtra las entradas y agrupa por estación usando groupingBy y counting.

b) Horas pico

Agrupa las entradas por hora del día y cuenta los registros de cada hora.

c) Rutas más utilizadas

Agrupa las entradas por ruta y permite ordenar el resultado para mostrar las rutas con más uso.

d) Patrones de viaje por usuario

Ordena las entradas cronológicamente y agrupa las estaciones visitadas por identificador de usuario.

e) Tiempo promedio entre estaciones

Calcula diferencias en minutos entre entradas consecutivas del mismo usuario, después de ordenarlas por fecha, y obtiene el promedio.

f) Sobrecarga de rutas

Simula la ocupación sumando entradas y restando salidas por ruta. Si el valor supera el umbral definido (100), la ruta se marca como CRITICA.

Funciones puras

Los métodos de ProcesadorTransporte reciben datos y producen resultados sin modificar la lista de entrada ni utilizar estado global. Para iguales datos y parámetros, el resultado es el mismo.

Inmutabilidad

Registro utiliza atributos privados y finales, no tiene setters y valida la acción en el constructor. El generador devuelve una lista no modificable. Los métodos de análisis producen colecciones nuevas y las exponen como no modificables cuando corresponde.

Funciones de orden superior y lambdas

filter, map, sorted, collect y groupingBy reciben funciones para filtrar, transformar, ordenar o agrupar. Se usan lambdas como r -> r.getAccion().equals("entrada") y referencias a métodos como Registro::getRuta.

Streams

Los Streams permiten encadenar operaciones declarativas sobre los registros, sin escribir ciclos que modifiquen la colección original. Se usa stream(); parallelStream() podría evaluarse en pruebas de rendimiento, manteniendo operaciones sin efectos secundarios.

Limitaciones de la simulación

Los 12.000 registros no son datos reales. El umbral de ocupación 100 es demostrativo y debe sustituirse por una capacidad validada. El promedio se calcula entre entradas consecutivas del mismo usuario; no representa necesariamente un viaje completo porque el caso no define cómo emparejar origen y destino.

Compilación y ejecución

Se requiere JDK 11 o superior. Desde la carpeta src ejecutar javac *.java y luego java Main. La compilación y ejecución se probaron correctamente; la salida y una captura de consola se incluyen en la carpeta evidencias.

Repositorio GitHub

Crear un repositorio llamado TecnoMovilData y subir src, evidencias, README.md y el informe. Verificar que la rama principal sea main. Enlace del repositorio: [Pegar aquí la URL del repositorio].

Conclusiones

Se aplicaron operaciones funcionales de Java para analizar registros de transporte urbano. Streams y lambdas facilitaron filtros, agrupaciones y ordenamientos de forma declarativa. La inmutabilidad ayuda a evitar modificaciones accidentales de datos compartidos.

También se observó que los indicadores dependen de las reglas del negocio. Para obtener cifras reales de ocupación y tiempos de viaje se necesitarían datos auténticos, capacidad de los vehículos y reglas claras para identificar trayectos.

Referencia

Institución Universitaria Digital de Antioquia. (2026). *Unidad 2: Caso de estudio. Procesamiento funcional de datos para un sistema de gestión de transporte urbano* [Material de curso].

Anexo A. Código fuente

El proyecto completo se entrega en el archivo ZIP. Se incluyen a continuación los cuatro archivos Java.

Registro.java

```
import java.time.LocalDateTime;
import java.util.Objects;

/**
 * Representa una validacion realizada por un pasajero en el sistema.
 * La clase es inmutable: sus atributos son privados y finales y no tiene setters.
 */
public final class Registro {
    private final String idUsuario;
    private final String ruta;
    private final String estacion;
    private final String accion;
    private final LocalDateTime timestamp;

    public Registro(String idUsuario, String ruta, String estacion,
                    String accion, LocalDateTime timestamp) {
        this.idUsuario = Objects.requireNonNull(idUsuario);
        this.ruta = Objects.requireNonNull(ruta);
        this.estacion = Objects.requireNonNull(estacion);
        this.accion = Objects.requireNonNull(accion).toLowerCase();
        this.timestamp = Objects.requireNonNull(timestamp);

        if (!this.accion.equals("entrada") && !this.accion.equals("salida")) {
            throw new IllegalArgumentException("La accion debe ser entrada o salida.");
        }
    }

    public String getIdUsuario() { return idUsuario; }
    public String getRuta() { return ruta; }
    public String getEstacion() { return estacion; }
    public String getAccion() { return accion; }
    public LocalDateTime getTimestamp() { return timestamp; }

    @Override
    public String toString() {
        return "Registro{" + idUsuario + ", " + ruta + ", " + estacion
                + ", " + accion + ", " + timestamp + "}";
    }
}
```

GeneradorDatos.java

```
import java.time.LocalDateTime;
import java.util.ArrayList;
import java.util.Collections;
import java.util.List;

/** Crea datos de prueba deterministas para poder repetir la demostracion. */
public final class GeneradorDatos {
    private GeneradorDatos() { }

    public static List<Registro> generar(int cantidad) {
        List<Registro> datos = new ArrayList<>();
        LocalDateTime inicio = LocalDateTime.of(2026, 11, 4, 5, 0);

        for (int i = 0; i < cantidad; i++) {
            int usuario = (i % 500) + 1;
            int ruta = (i % 12) + 1;
            int estacion = ((i * 7) % 20) + 1;
            // Se alternan entradas y salidas para representar validaciones distintas.
            String accion = (i % 3 == 0) ? "salida" : "entrada";
            LocalDateTime fecha = inicio.plusMinutes((i * 7L) % (15 * 60));
            datos.add(new Registro(
                    String.format("U%03d", usuario),
                    String.format("R%02d", ruta),
                    String.format("E%02d", estacion),
                    accion,
                    fecha
            ));
        }
        return Collections.unmodifiableList(datos);
    }
}
```

ProcesadorTransporte.java

```
import java.time.Duration;
import java.util.AbstractMap;
import java.util.Collections;
import java.util.Comparator;
import java.util.List;
import java.util.Map;
import java.util.stream.Collectors;

/**
 * Operaciones declarativas. Cada metodo recibe datos y devuelve un resultado
 * nuevo sin modificar la coleccion recibida ni mantener estado compartido.
```

```
 */
public final class ProcesadorTransporte {
    private ProcesadorTransporte() { }

    // a) Cantidad de entradas por estacion.
    public static Map<String, Long> afluenciaPorEstacion(List<Registro> registros) {
        return Collections.unmodifiableMap(registros.stream()
                .filter(r -> r.getAccion().equals("entrada"))
                .collect(Collectors.groupingBy(
                        Registro::getEstacion, Collectors.counting())));
    }

    // b) Cantidad de entradas agrupadas por hora.
    public static Map<Integer, Long> horasPico(List<Registro> registros) {
        return Collections.unmodifiableMap(registros.stream()
                .filter(r -> r.getAccion().equals("entrada"))
                .collect(Collectors.groupingBy(
                        r -> r.getTimestamp().getHour(),
                        Collectors.counting())));
    }

    // c) Volumen de uso por ruta, contando las entradas.
    public static Map<String, Long> rutasMasUtilizadas(List<Registro> registros) {
        return Collections.unmodifiableMap(registros.stream()
                .filter(r -> r.getAccion().equals("entrada"))
                .collect(Collectors.groupingBy(
                        Registro::getRuta, Collectors.counting())));
    }

    // d) Estaciones visitadas por usuario, en orden cronologico.
    public static Map<String, List<String>> patronesPorUsuario(List<Registro> registros) {
        Map<String, List<String>> resultado = registros.stream()
                .filter(r -> r.getAccion().equals("entrada"))
                .sorted(Comparator.comparing(Registro::getTimestamp))
                .collect(Collectors.groupingBy(
                        Registro::getIdUsuario,
                        Collectors.mapping(Registro::getEstacion, Collectors.toList())));
        resultado.replaceAll((usuario, estaciones) ->
                Collections.unmodifiableList(estaciones));
        return Collections.unmodifiableMap(resultado);
    }

    // e) Promedio en minutos entre entradas consecutivas de cada usuario.
    public static double tiempoPromedioEntreEstaciones(List<Registro> registros) {
        List<Registro> ordenados = registros.stream()
                .filter(r -> r.getAccion().equals("entrada"))
                .sorted(Comparator.comparing(Registro::getIdUsuario)
                        .thenComparing(Registro::getTimestamp))
                .collect(Collectors.toList());

        List<Long> diferencias = java.util.stream.IntStream.range(1, ordenados.size())
                .filter(i -> ordenados.get(i - 1).getIdUsuario()
                        .equals(ordenados.get(i).getIdUsuario()))
                .mapToObj(i -> Duration.between(
                        ordenados.get(i - 1).getTimestamp(),
                        ordenados.get(i).getTimestamp()).toMinutes())
                .filter(minutos -> minutos >= 0)
                .collect(Collectors.toList());

        return diferencias.isEmpty() ? 0.0
                : diferencias.stream().mapToLong(Long::longValue).average().orElse(0.0);
    }

    // f) Ocupacion simulada: entradas menos salidas por ruta.
    public static Map<String, String> detectarRutasCriticas(
            List<Registro> registros, long umbral) {
        Map<String, Long> ocupacion = registros.stream()
                .collect(Collectors.groupingBy(
                        Registro::getRuta,
                        Collectors.summingLong(r ->
                                r.getAccion().equals("entrada") ? 1L : -1L)));
        return Collections.unmodifiableMap(ocupacion.entrySet().stream()
                .collect(Collectors.toMap(
                        Map.Entry::getKey,
                        e -> e.getValue() > umbral ? "CRITICA" : "Normal")));
    }

    public static List<Map.Entry<String, Long>> topRutas(
            Map<String, Long> rutas, int limite) {
        return rutas.entrySet().stream()
                .sorted(Map.Entry.<String, Long>comparingByValue().reversed())
                .limit(limite)
                .map(e -> new AbstractMap.SimpleImmutableEntry<>(e.getKey(), e.getValue()))
                .collect(Collectors.toList());
    }
}
```

Main.java

```
import java.util.List;
import java.util.Map;

public class Main {
    public static void main(String[] args) {
        int cantidad = 12000;
        List<Registro> registros = GeneradorDatos.generar(cantidad);
```

```
        System.out.println("=== TECNOMOVIL DATA ===");
        System.out.println("Registros procesados: " + registros.size());

        Map<String, Long> estaciones =
                ProcesadorTransporte.afluenciaPorEstacion(registros);
        System.out.println("\n1. Afluencia por estacion (primeras 5):");
        estaciones.entrySet().stream().limit(5)
                .forEach(e -> System.out.println(e.getKey() + ": " + e.getValue()));

        Map<Integer, Long> horas = ProcesadorTransporte.horasPico(registros);
        System.out.println("\n2. Flujo por hora:");
        horas.entrySet().stream().sorted(Map.Entry.comparingByKey())
                .forEach(e -> System.out.println(e.getKey() + ":00 -> " + e.getValue()));

        Map<String, Long> rutas = ProcesadorTransporte.rutasMasUtilizadas(registros);
        System.out.println("\n3. Tres rutas mas utilizadas:");
        ProcesadorTransporte.topRutas(rutas, 3)
                .forEach(e -> System.out.println(e.getKey() + ": " + e.getValue()));

        Map<String, List<String>> patrones =
                ProcesadorTransporte.patronesPorUsuario(registros);
        System.out.println("\n4. Patron de viaje de U001:");
        System.out.println(patrones.getOrDefault("U001", List.of()));

        double promedio =
                ProcesadorTransporte.tiempoPromedioEntreEstaciones(registros);
        System.out.printf("\n5. Tiempo promedio entre entradas consecutivas: %.2f minutos%n",
                promedio);

        Map<String, String> estado =
                ProcesadorTransporte.detectarRutasCriticas(registros, 100);
        System.out.println("\n6. Estado de rutas (umbral simulado: 100):");
        estado.entrySet().stream().sorted(Map.Entry.comparingByKey())
                .forEach(e -> System.out.println(e.getKey() + ": " + e.getValue()));

        System.out.println("\nProceso terminado. La lista original no fue modificada.");
    }
}
```